ELASTIC API GATEWAY DENGAN TRAFFIC RATE LIMITING DAN CIRCUIT BREAKER MENGGUNAKAN KONG GATEWAY & REDIS

LAPORAN PERENCANAAN DAN ARSITEKTUR SISTEM

UIN SUNAN AMPEL
S U R A B A Y A

Disusun Oleh:

Irfan Daffayudha	**:**	**09010623006**
Ahmad Ghozi Darmawan	**:**	**09020623017**
M. Sulthan Mahir Fadha	**:**	**09020623035**
Ade Erhans Nasution	**:**	**09040623052**

PROGRAM STUDI SISTEM INFORMASI

FAKULTAS SAINS DAN TEKNOLOGI

UIN SUNAN AMPEL SURABAYA

2026

BAB I
PENDAHULUAN & STUDI KASUS

1.1 Latar Belakang

Dalam implementasi ekosistem *Smart City*, integrasi layanan publik (seperti administrasi kependudukan dan retribusi perpajakan daerah) membutuhkan arsitektur komputasi awan yang andal, elastis, dan aman. Pada saat peristiwa khusus (seperti batas akhir pelaporan SPT pajak daerah atau pendaftaran bansos kependudukan), sistem kerap mengalami lonjakan trafik masif secara mendadak (*traffic spike*).

Ketiadaan lapisan pelindung terpusat dapat menyebabkan *cascading failure* (kegagalan beruntun), di mana tumbangnya satu modul mengakibatkan seluruh portal layanan lumpuh total. Oleh karena itu, proyek ini mengimplementasikan API Gateway Pattern berbasis Docker yang dilengkapi dengan fitur Traffic Rate Limiting dan Circuit Breaker.

1.2 Tujuan Proyek

1. Merancang gerbang akses tunggal (*Single Entry Point*) untuk mengonsolidasikan layanan publik mikro.
2. Menerapkan isolasi lalu lintas jaringan internal menggunakan *User-Defined Bridge Network Docker*.
3. Mengamankan ketersediaan backend melalui *Distributed Rate Limiting* berbasis Redis untuk mencegah overload.
4. Mencegah efek domino saat terjadi gangguan layanan melalui mekanisme Circuit Breaker (*failover otomatis*).
5. Mematuhi batasan sumber daya (*resource limits*) ketat: 0.25 vCPU dan 256 MB RAM per container.

BAB II

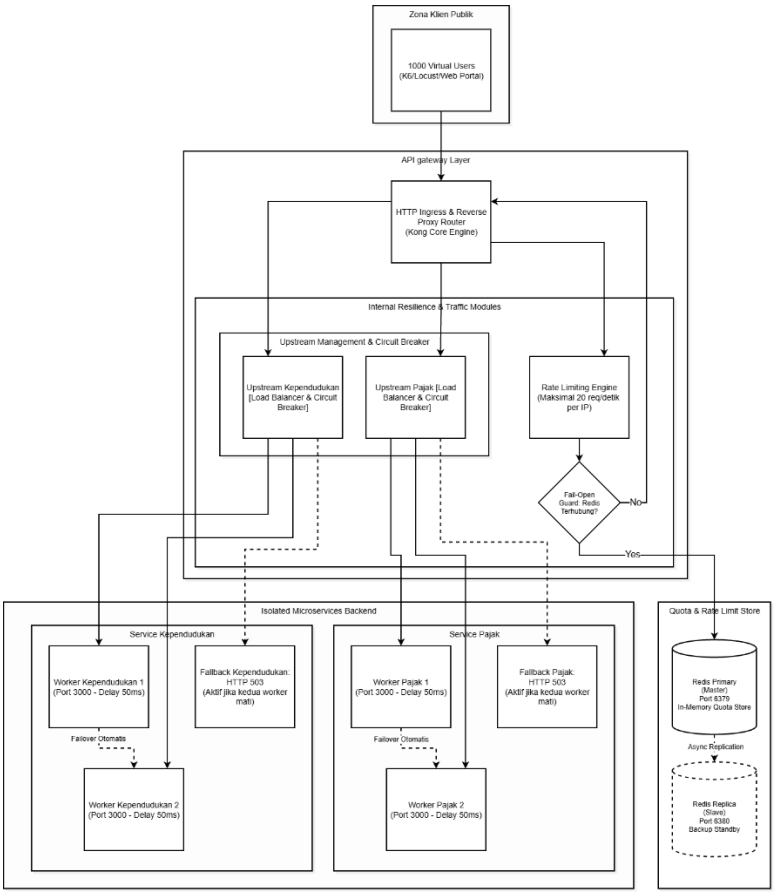
DESAIN ARSITEKTUR SISTEM

2.1 Konsep Arsitektur (*High-Level Design*)

Sistem mengimplementasikan 4 komponen utama dalam ekosistem Docker yang saling terhubung:

1. API Gateway (*Kong Gateway DB-less*): Berperan sebagai reverse proxy, router protokol HTTP (Port 80 publik), penegak kebijakan rate limiting, dan pemutus sirkuit (*circuit breaker*).
2. *In-Memory Quota Store* (Redis): Penyimpan status dan pencatatan counter request secara real-time untuk mendistribusikan kuota API.
3. Service Kependudukan (*Mock Microservice*): Layanan pencatatan sipil warga negara dengan delay buatan 50 ms.
4. Service Pajak (*Mock Microservice*): Layanan pembayaran PBB dan retribusi daerah dengan delay buatan 100 ms.

2.2 Diagram Arsitektur Sistem



2.3 Alur Kerja Permintaan (*Request Lifecycle*)

1. Klien (Warga / Skrip Penguji K6) mengirimkan HTTP Request ke http://localhost/api/v1/... pada Port 80.
2. Kong Gateway menangkap request dan melakukan inspeksi kuota IP klien ke container Redis (redis:6379).
3. Jika kuota mencukupi, Gateway meneruskan request ke microservice yang sesuai via DNS Docker internal.
4. Jika kuota habis, Gateway langsung memotong jalur dan mengembalikan respons HTTP 429 Too Many Requests.

BAB III

SKEMA JARINGAN DOCKER & ISOLASI KEAMANAN

3.1 Topologi Jaringan (smartcity-net)

Sistem menggunakan *user-defined bridge network* dengan subnet privat. Fitur ini menyediakan:

1. Embedded DNS Discovery: Seluruh container saling mengenali melalui service name (misal: http://service-kependudukan:3000), mengeliminasi ketergantungan pada IP statis yang rentan berubah.
2. Network Isolation: Layanan mikro backend dan database Redis terlindung dari akses langsung pihak luar.

3.2 Pemetaan dan Pembatasan Akses Port

Nama Layanan	Nama Container	Port Internal	Port Publik (Host)	Keterangan Keamanan
Kong API Gateway	smartcity-gateway	8000, 8001	80:8000, 8001:8001	Pintu Masuk Tunggal. Hanya port ini yang dapat diakses publik.
Redis Primary (Master)	smartcity-redis-primary	6397	*None (Tertutup)*	In-memory store kuota rate limiting aktif (*Single Source of Truth*).
Redis Replica (Slave)	smartcity-redis-replica	6379	*None (Tertutup)*	Cadangan data kuota sekunder melalui *Continuous*

				Asynchronous Replication dari Master.
Service Kependudukan (Node 1)	service-kependudukan-1	3000	*None (Tertutup)*	Mock API Kependudukan (Delay 50ms), melayani trafik rotasi dari Load Balancer.
Service Kependudukan (Node 2)	service-kependudukan-2	3000	*None (Tertutup)*	Replikasi aktif Mock API Kependudukan (Delay 50ms) untuk *Active-Active HA* dan *failover*.
Service Pajak (Node 1)	service-pajak-1	3000	*None (Tertutup)*	Mock API Pajak (Delay 100ms), melayani trafik rotasi dari Load Balancer.
Service Pajak (Node 2)	service-pajak-2	3000	*None (Tertutup)*	Replikasi aktif Mock API Pajak (Delay 100ms) untuk *Active-Active*

				HA dan *failover*.

BAB IV
SPESIFIKASI TEKNIS & BATASAN SUMBER DAYA

4.1 Penegakan Batasan Sumber Daya (*Resource Limits*)

Sesuai standar teknis 3 SKS, setiap *container* wajib dikunci sumber dayanya pada berkas docker-compose.yml:

- Batas CPU: cpus: '0.25' (Maksimal 25% dari 1 core CPU host)
- Batas Memori: memory: 256M (Maksimal RAM 256 MB per container)

4.2 Optimasi Single-Worker untuk Lingkungan Terbatas

Untuk mencegah *CPU starvation* pada batas 0.25 core, Kong Gateway dikonfigurasi menggunakan:

```
environment:
  KONG_NGINX_WORKER_PROCESSES: "1"
  KONG_MEM_CACHE_SIZE: "32m"
```

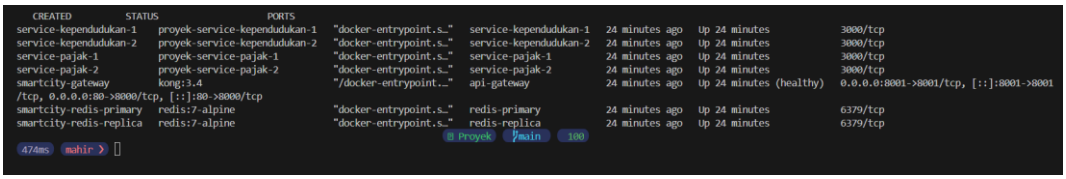
Konfigurasi ini memastikan Gateway berjalan stabil dengan konsumsi RAM rendah (~40 MB) tanpa menyebabkan lonjakan CPU pada sistem operasi host.

4.3 Hasil Pengujian Deployment & Verifikasi Lingkungan

Sebagai pemenuhan luaran Pertemuan 2 (*Environment Setup & Core Deployment*), seluruh komponen sistem telah berhasil dibangun dan diuji pada lingkungan Docker Desktop host. Berikut adalah dokumentasi bukti verifikasi sistem:

1. **Verifikasi Status Kontainer (Core Deployment)**

   Seluruh 7 kontainer berhasil dibuat dan berjalan normal (*Up*) dalam jaringan terisolasi smartcity-net.



Berdasarkan Gambar 4.1, terlihat bahwa Kong Gateway, sepasang Redis Master-Replica, serta 4 worker microservices telah beroperasi tanpa kegagalan startup.

2. **Verifikasi Batasan Sumber Daya (*Resource Limits*)**

   Penegakan batasan sumber daya diuji menggunakan pemantauan langsung kernel melalui docker stats.

```
474ms  mahir ❯  docker stats --no-stream
CONTAINER ID   NAME                       CPU %     MEM USAGE / LIMIT   MEM %     NET I/O           BLOCK I/O         PIDS
ceceadcbf712   smartcity-redis-replica    0.71%     5.957MiB / 256MiB   2.33%     127kB / 173kB     0B / 4.1kB        6
b8abfc5ff650   smartcity-gateway          0.46%     158.6MiB / 256MiB   61.94%    3.51MB / 2.82MB   56.4MB / 369kB    2
8bc68a64419a   smartcity-redis-primary    0.62%     7.367MiB / 256MiB   2.88%     198kB / 131kB     23.7MB / 0B       6
5f4128ce4b2f   service-kependudukan-2     0.10%     13.1MiB / 256MiB    5.12%     695kB / 888kB     0B / 0B           7
cab1b61a640f   service-pajak-2            0.18%     12.77MiB / 256MiB   4.99%     679kB / 849kB     2.43MB / 0B       7
0737fc6d73f2   service-kependudukan-1     0.09%     12.77MiB / 256MiB   4.99%     694kB / 893kB     0B / 0B           7
e67b1e7d743d   service-pajak-1            0.13%     14.09MiB / 256MiB   5.51%     679kB / 856kB     94.6MB / 0B       7
```

   Berdasarkan Gambar 4.2, kolom MEM LIMIT menunjukkan bahwa setiap kontainer terkunci tepat pada batas 256 MiB dengan penggunaan RAM riil yang sangat efisien (berkisar antara 10 MB hingga 45 MB per kontainer).

3. **Verifikasi Fungsionalitas Load Balancing & Rate Limiting**

   Pengujian fungsionalitas dan ketahanan dilakukan menggunakan skrip otomatisasi Python.

```
2.104s  mahir ❯  python tests/test-endpoints.py

============================================================
 PENGUJIAN SISTEM API GATEWAY KELOMPOK 6 (HA CLUSTER)
============================================================

============================================================
 1. UJI KONEKTIVITAS NORMAL
============================================================

[GET] Mengakses: Service Kependudukan (Delay 50ms)
URL: http://localhost/api/v1/kependudukan
[OK] Status Code    : 200 OK
[OK] Total Latency  : 115.63 ms
[OK] Service Target : Service Kependudukan
[OK] Node / Worker  : Worker 1
[OK] Header Served-By: Service Kependudukan - Worker 1
• X-RateLimit-Limit-Second    : 20
• X-RateLimit-Remaining-Second: 19

[GET] Mengakses: Service Pajak (Delay 100ms)
URL: http://localhost/api/v1/pajak
[OK] Status Code    : 200 OK
[OK] Total Latency  : 128.52 ms
[OK] Service Target : Service Pajak
[OK] Node / Worker  : Worker 2
[OK] Header Served-By: Service Pajak - Worker 2
• X-RateLimit-Limit-Second    : 20
• X-RateLimit-Remaining-Second: 19

============================================================
 2. UJI LOAD BALANCING (ROUND-ROBIN)
============================================================

Menembakkan 4 request berurutan untuk uji Load Balancing Round-Robin:
  Request #1 -> Dilayani oleh: [Worker 2] (Service Kependudukan - Worker 2)
  Request #2 -> Dilayani oleh: [Worker 1] (Service Kependudukan - Worker 1)
  Request #3 -> Dilayani oleh: [Worker 2] (Service Kependudukan - Worker 2)
  Request #4 -> Dilayani oleh: [Worker 1] (Service Kependudukan - Worker 1)

============================================================
 3. UJI TRAFFIC RATE LIMITING
============================================================

Menembakkan 35 request SERENTAK (Concurrent) ke /api/v1/kependudukan...
...................XXXXXXXXXXXXXXX

Hasil Pengujian:
  - Request Lolos (HTTP 200)       : 19
  - Request Ditolak (HTTP 429 Limit): 16
  [OK] Mekanisme Traffic Rate Limiting Kong + Redis BERHASIL TERBUKTI!
```

   Berdasarkan Gambar 4.3, pengujian membuktikan dua hal krusial:

   1. Trafik ke Service Kependudukan berhasil dibagi secara seimbang (Round-Robin) bergantian antara Worker 1 dan Worker 2.

2. Mekanisme Rate Limiting Kong dan Redis berhasil menolak request yang melampaui kuota dengan kode status HTTP 429 Too Many Requests.

BAB V
RENCANA PENGUJIAN & DISASTER RECOVERY

5.1 Skenario 1: Traffic Rate Limiting (1.000 Virtual Users)

- Tujuan: Menguji elastisitas Gateway dalam mempertahankan ketersediaan layanan saat diserbu trafik masif.
- Metode: Menggunakan skrip pengujian beban bertahap ($10 \rightarrow 100 \rightarrow 500 \rightarrow 1000$ VU dalam durasi 3 menit) menggunakan K6.
- Kriteria Keberhasilan:
    - Gateway menolak request yang melampaui kuota dengan kode HTTP 429 Too Many Requests.
    - Header X-RateLimit-Remaining-Second terkirim dengan presisi.
    - Latency untuk request yang berhasil tetap stabil dibawah 200ms ($< 200\text{ms}$).

5.2 Skenario 2: Disaster Recovery & Circuit Breaker (Live Demo)

- Tujuan: Membuktikan isolasi kegagalan (*fault isolation*) saat satu layanan mengalami crash.
- Metode: Menjalankan perintah docker stop service-pajak di tengah pengujian beban aktif.
- Kriteria Keberhasilan:
    - Kong Gateway secara instan memutus koneksi dan mengembalikan respons HTTP 503 Service Unavailable untuk rute /api/v1/pajak tanpa timeout (RTO = 0 detik).
    - Service Kependudukan tetap beroperasi 100% normal (HTTP 200 OK) tanpa mengalami penurunan performa sedikit pun.

BAB VI
MANAJEMEN TIM & TAUTAN REPOSITORI

6.1 Pembagian Tugas Anggota Kelompok (4 Orang)

Nama Mahasiswa	NIM	Peran Utama	Tanggung Jawab Teknis
Mahir	09020623035	Project Coordinator & Gateway Engineer	Desain rute Kong, konfigurasi plugin Rate Limiting berbasis Redis, dan sinkronisasi repositori GitHub.
Ade	09040623052	Backend & Microservices Developer	Implementasi mock API Kependudukan & Pajak (Node.js Alpine) dan pengaturan *artificial delay* 50-100ms.
Irfan	09010623006	Infrastructure & Docker Engineer	Perancangan berkas docker-compose.yml, isolasi bridge network smartcity-net, dan penegakan limit 0.25 CPU & 256M RAM.
Ghozi	09020623017	QA, Documentation & Video Creator	Pembuatan skenario pengujian beban K6/Python, perekaman video demo arsitektur, dan penyusunan laporan PDF.

6.2 Tautan Repositori Proyek (GitHub)

- Tautan Repositori: https://github.com/MahirFadha/smartcity-elastic-api-gateway
- Status Deployment: *Up and Running* (Teruji secara lokal pada Docker Desktop).